🌲 DAY 1: MASTER ARROW WORKFLOWS & COMPONENT EXECUTION TREES

The Definitive Topic-by-Topic Visual Architecture Guide (Concept ──► Hardware ──► Command ──► Verification)

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 1 (Full 6 Hours: 3h Morning Architecture + 3h Afternoon Hands-On Lab)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

🗺 MASTER INDEX OF DAY 1 ARROW TREES

- Topic 1: Introduction to HPC & Scientific Parallelism Workflow
- Topic 2: Intel HPC Hardware Architecture Continuum
  - 2A. Intel Xeon CPU Architecture & Silicon Pipeline Flow
  - 2B. Intel Data Center GPU (Xe-HPC Max Series) Execution Flow
  - 2C. Habana Gaudi (Gaudi 2 & 3) AI Accelerator Architecture Flow
  - 2D. Memory Hierarchy & Silicon Latency Wall Spectrum
  - 2E. Workload-Driven Hardware Selection Decision Matrix Tree
- Topic 3: Linux Fundamentals & HPC Virtual File System (VFS) Workflow
- Topic 4: Shell Scripting & Automated HPC Job Pipeline Flow
- Topic 5: Secured Shell (SSH) Cryptographic Handshake & Key-Based Access Flow
- Topic 6: Intel oneAPI Heterogeneous Compute Pipeline (SYCL + USM + Level Zero)
- Topic 7: Intel HPC Toolkit Cluster Scaling & Profiling Loop (MPI + VTune)
- Topic 8: The Unified End-to-End Supercomputer Execution Lifecycle

---

Topic 1: Introduction to HPC & Scientific Parallelism Workflow

1.1 Arrow-Based Architectural Flow

```
[ Massive Scientific / AI Problem Space ]
(e.g., 100M Equations, 10M Row Preprocessing, 70B Parameter LLM)
       |
       ▼
[ Complexity & Feasibility Audit ]
       ├── Single CPU Core Execution Time: 60 Hours (Infeasible for Production)
       └── Target Execution Time: Under 15 Minutes (HPC Acceleration Required)
       |
       ▼
[ Mathematical Problem Decomposition ]
       ├── Domain Decomposition ──► Split Spatial Grid / Matrix into Sub-blocks
       └── Functional Decomposition ──► Ingestion ──► Computation ──► Checkpointing
       |
       ▼
[ Parallel Computing Model Selection ]
       ├── Shared-Memory Parallelism (OpenMP) ──► Low latency, single-node RAM
       └── Distributed-Memory Parallelism (MPI) ──► Network messaging across compute nodes
       |
       ▼
[ Concurrent Hardware Execution ]
       ├── Core 0 / Rank 0: Computes Sub-Block A ──┐
       ├── Core 1 / Rank 1: Computes Sub-Block B ──┼──► [ Concurrent Compute Cycles ]
       └── Core N / Rank N: Computes Sub-Block N ──┘
       |
       ▼
[ Synchronization & Collective Reduction ]
       └── Partial Sums / Gradients aggregated via MPI_Allreduce() or Ring-Reduction
       |
       ▼
[ Ground-Truth Verification & Speedup Calculation ]
       ├── Correctness: Bit-exact numerical check against serial baseline
       ├── Speedup (S) = T_serial / T_parallel  (e.g., 60 min / 12 min = 5.0× Speedup)
       └── Parallel Efficiency (E) = Speedup / N_cores (e.g., 5.0 / 8 = 62.5% Efficiency)
```

1.2 Core Theory & First Principles

- **Amdahl's Law**: $S(N) = \frac{1}{(1-P) + \frac{P}{N}}$. Theoretical speedup is strictly bounded by the sequential portion $(1 - P)$ of the algorithm. If 5% of code is serial, maximum speedup on an infinite number of processors cannot exceed 20×.
- **Gustafson-Barsis Law**: $S(N) = N - \alpha(N - 1)$. As computational power scales, problem size expands to utilize memory, maintaining high parallel efficiency.

1.3 Practical Command & Ground-Truth Verification

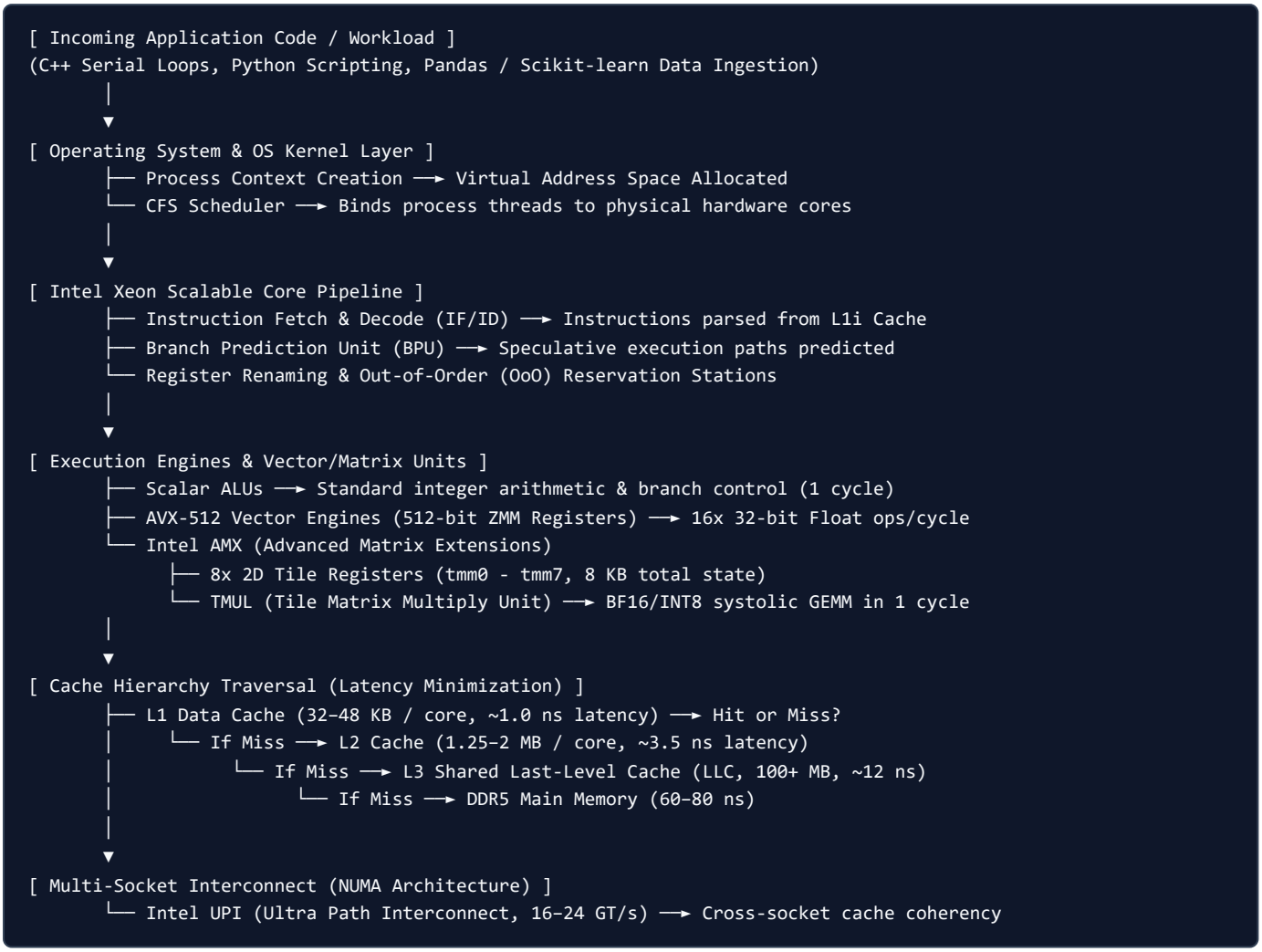
```
# Verify available hardware parallelism on current node
lscpu | grep -E "Model name|Socket\(s\)|Core\(s\) per socket|Thread\(s\) per core"
```

Expected Output:

```
Model name:            Intel(R) Xeon(R) Platinum 8480+ (or Haswell/Broadwell in Colab)
Socket(s):             2
Core(s) per socket:    56
Thread(s) per core:    2
```

Topic 2: Intel HPC Hardware Architecture Continuum

2A. Intel Xeon CPU Architecture & Silicon Pipeline Flow



2B. Intel Data Center GPU (Xe-HPC Max Series) Execution Flow

```
[ Deep Learning / High-Density Matrix Workload ]
(Dense GEMM: C = A × B, 2D/3D FFT, Convolutional Tensors, SYCL Kernels)
       │
       ▼
[ Host CPU Command Submission ]
       ├── Host allocates Unified Shared Memory (USM) or Device VRAM
       └── Host queues kernel commands via Intel Level Zero (`ze_loader`) / OpenCL
       │
       ▼
[ PCIe Gen 5 / CXL Bus Transfer ] ──► (Data DMA stream into GPU onboard VRAM)
       │
       ▼
[ Intel Data Center GPU Max 1550 (Ponte Vecchio Architecture) ]
       │
       ├── Compute Tile 0 ──┐ (Connected via High-Bandwidth Foveros 3D Stacking)
       └── Compute Tile 1 ──┘
             │
             ▼
       [ Xe-HPC Core Array (Up to 128 Xe Cores) ]
             ├── Vector Engines (256-bit SIMD ALUs) ──► High-throughput vector math
             └── XMX (Xe Matrix Extensions) ──► Hardware Systolic Arrays
                   ├── FP64 / FP32 / FP16 / BF16 / INT8 Tensor Operations
                   └── Multiplies 4096-bit matrix fragments per clock cycle
             │
             ▼
       [ High-Bandwidth GPU Memory Subsystem ]
             ├── 64 KB Shared Local Memory (SLM) per Xe Core (sub-nanosecond)
             ├── 408 MB Massive On-Die L2 Cache (Ultra-high throughput crossbar)
             └── Up to 128 GB HBM2e (High-Bandwidth Stacked Memory > 3.2 TB/s Bandwidth)
       │
       ▼
[ Multi-GPU Scaling Fabric ]
       └── Intel XeLink Bridges (Direct GPU-to-GPU coherent fabric, bypasses CPU)
```

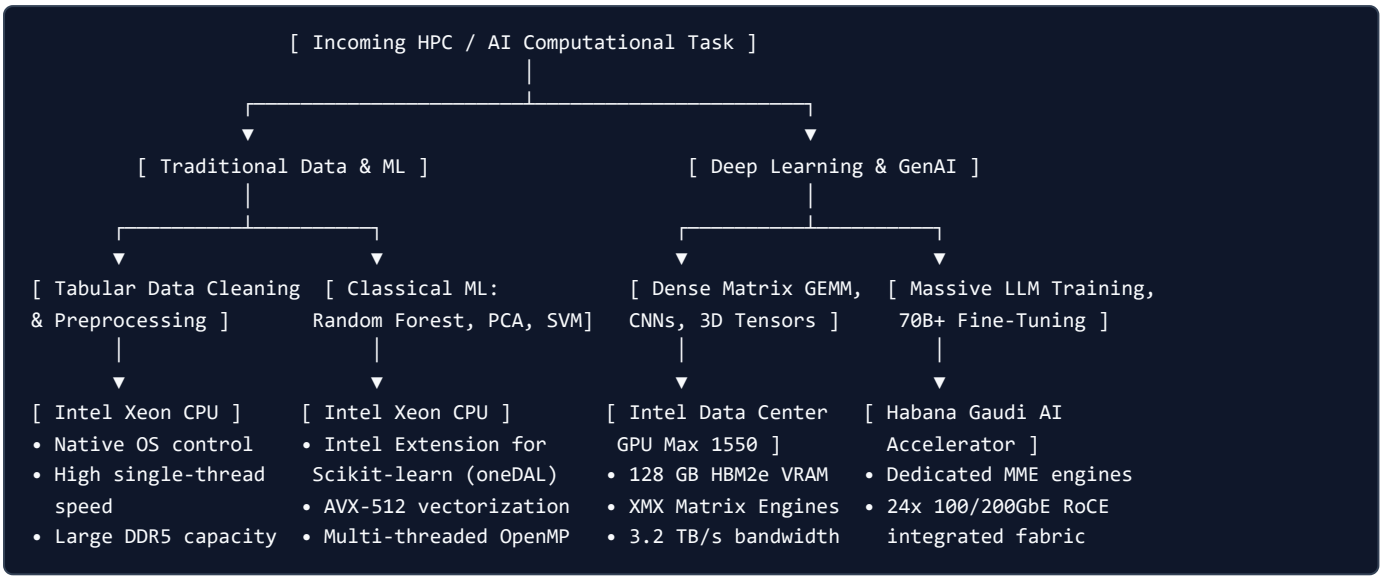
2C. Habana Gaudi (Gaudi 2 & 3) AI Accelerator Architecture Flow

```
[ Generative AI & Large Language Model (LLM) Workload ]
(Llama 70B, GPT Fine-Tuning, Transformer Attention Layers, Multimodal Diffusion)
       │
       ▼
[ Host Orchestrator (Intel Xeon Server) ]
       ├── PyTorch / Hugging Face model loaded
       └── Habana SynapseAI Software Stack & Graph Compiler intercepts operators
       │
       ▼
[ SynapseAI Graph Compiler Optimization ]
       ├── Computational Graph Optimization (Node Fusion: Conv + BatchNorm + GELU)
       ├── Mixed Precision Quantization (FP8 / BF16 / FP16)
       └── Automated Memory Layout & Buffer Allocation
       │
       ▼
[ Habana Gaudi Hardware Execution Engines ]
       ├── Matrix Multiplication Engine (MME)
       │     └── Massive dedicated systolic array for 3D GEMM & 2D Convolution
       │
       └── Tensor Processor Cores (TPC Array: Fully Programmable VLIW SIMD)
             ├── Custom ISA engineered strictly for AI math
             └── Executes non-linear ops: Softmax, LayerNorm, GeLU, RoPE embeddings
       │
       ▼
[ Memory Subsystem ]
       └── 96 GB (Gaudi 2) / 128 GB (Gaudi 3) HBM2e (> 2.45 TB/s Bandwidth)
       │
       ▼
[ Native Scale-Out Networking (The Habana Architectural Superpower) ]
       ├── 24x Integrated 100GbE / 200GbE RoCE v2 Ports Directly on Silicon Die
       ├── ZERO External NICs (No InfiniBand Host Channel Adapters needed!)
       └── Direct All-to-All Cluster Fabric Interconnect between Gaudi Accelerators
```

2D. Memory Hierarchy & Silicon Latency Wall Spectrum

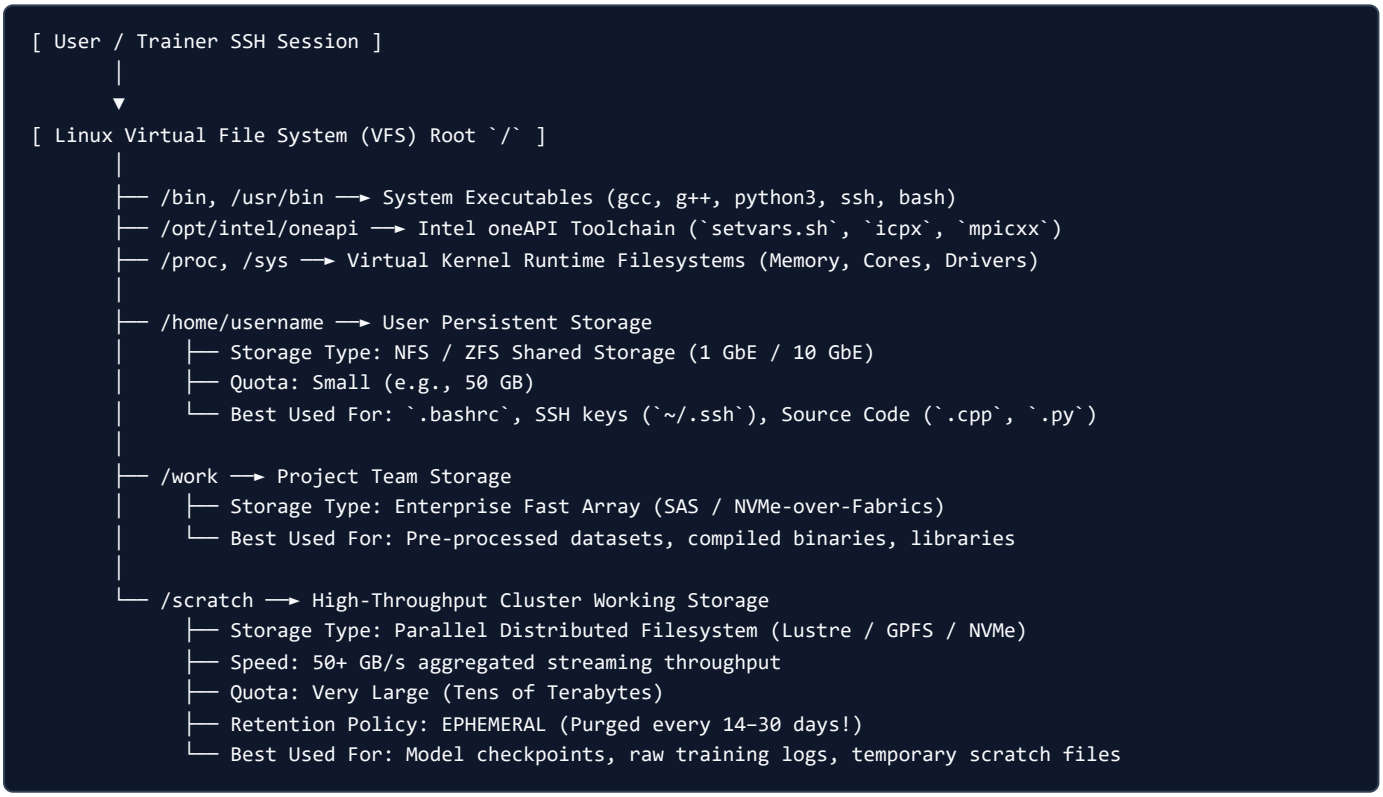
```
[ CPU Registers ] ──────────────► Latency: 0.3 ns  | Bandwidth: ~10,000 GB/s (Fastest)
       │
       ▼ (Hits Cache Wall)
[ L1d Cache (32–48 KB) ] ──────► Latency: 1.0 ns  | Bandwidth: ~3,000 GB/s
       │
       ▼
[ L2 Cache (1.25–2 MB) ] ──────► Latency: 3.5 ns  | Bandwidth: ~1,500 GB/s
       │
       ▼
[ L3 Shared LLC (100+ MB) ] ──► Latency: 12.0 ns | Bandwidth: ~800 GB/s
       │
       ▼ (Hits DRAM Wall)
[ DDR5 Main Memory (RAM) ] ──► Latency: 65–80 ns| Bandwidth: ~300 GB/s (300× Slower than L1)
       │
       ▼ (Hits Socket Boundary)
[ Remote NUMA Socket RAM ] ──► Latency: 140 ns  | Bandwidth: Limited by UPI Link
       │
       ▼ (Hits PCIe Bus)
[ NVMe PCIe 5.0 SSD ] ─────────► Latency: 10–25 µs| Bandwidth: ~14 GB/s (10,000× Slower)
       │
       ▼ (Hits Network Switch)
[ Cluster InfiniBand / RoCE ] ─► Latency: 1–2 µs  | Bandwidth: 200–400 Gbps Fabric
```

2E. Workload-Driven Hardware Selection Decision Matrix Tree



Topic 3: Linux Fundamentals & HPC Virtual File System (VFS) Workflow

3.1 VFS Hierarchy Arrow Flow



3.2 Linux Permission Matrix Flow (`chmod 755 / 644 / 600`)

```
[ Octal Digit ] ──► [ Binary Bitmask ] ──► [ Permission Capability ]
       7        ──►      111 (4+2+1)   ──► Read + Write + Execute (`rwx`)
       6        ──►      110 (4+2+0)   ──► Read + Write (`rw-`)
       5        ──►      101 (4+0+1)   ──► Read + Execute (`r-x`)
       4        ──►      100 (4+0+0)   ──► Read Only (`r--`)
       0        ──►      000 (0+0+0)   ──► Zero Access (`---`)

[chmod 755 run.sh]        ──► User: rwx (7) | Group: r-x (5) | Others: r-x (5) [Executables]
[chmod 644 config.env]    ──► User: rw- (6) | Group: r-- (4) | Others: r-- (4) [Shared Configs]
[chmod 600 ~/.ssh/id_ed]  ──► User: rw- (6) | Group: --- (0) | Others: --- (0) [Private Keys]
```

3.3 Practical Linux Navigation Commands & Verification

```
# 1. Create standardized high-performance project hierarchy
mkdir -p ~/hpc_day01/{scripts,data,logs,bin}

# 2. Verify directory creation and permissions
ls -ld ~/hpc_day01/*
```

Expected Output:

```
drwxr-xr-x 2 student student 4096 Oct 25 10:00 bin
drwxr-xr-x 2 student student 4096 Oct 25 10:00 data
drwxr-xr-x 2 student student 4096 Oct 25 10:00 logs
drwxr-xr-x 2 student student 4096 Oct 25 10:00 scripts
```

Topic 4: Shell Scripting & Automated HPC Job Pipeline Flow

4.1 Automated Script Execution Pipeline Arrow Workflow

```
[ Developer Writes Shell Script (`hpc_node_audit.sh`) ]
       |
       ▼
[ Shebang Interpretation (`#!/bin/bash`) ] ──► Kernel selects /bin/bash execution binary
       |
       ▼
[ Strict Pipeline Safety Flags (`set -eo pipefail`) ]
       ├── `set -e` ──► Immediate termination if any single command exits non-zero (fails)
       ├── `set -o pipefail` ──► Fails if any command in a pipeline (`|`) returns error
       └── Prevents silent data corruption in multi-step HPC runs
       |
       ▼
[ Environment Variable Extraction ]
       ├── `export TIMESTAMP=$(date +%Y%m%d_%H%M%S)` ──► Dynamic run identifier
       └── `export OMP_NUM_THREADS=$(nproc)` ──► Binds thread pool to available cores
       |
       ▼
[ System Diagnostics & Probing ]
       ├── `lscpu | grep "Model name"` ──► Captures CPU microarchitecture
       ├── `free -h` ──► Probes available memory headroom
       └── `lscpu | grep -E "avx512|amx"` ──► Validates hardware vector flags
       |
       ▼
[ Output Redirection & Logging ]
       ├── `tee -a ~/hpc_day01/logs/audit_${TIMESTAMP}.log` ──► Console + File logging
       └── Returns clean exit status: `exit 0`
```

4.2 Script Architecture & Execution Code

```
cat << 'EOF' > ~/hpc_day01/scripts/hpc_node_audit.sh
#!/bin/bash
set -eo pipefail

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
LOG_FILE=~/hpc_day01/logs/audit_${TIMESTAMP}.log

echo "================================================" | tee -a "${LOG_FILE}"
echo "    AUTOMATED HPC CLUSTER NODE AUDIT REPORT      " | tee -a "${LOG_FILE}"
echo "    Generated on: $(date)                        " | tee -a "${LOG_FILE}"
echo "================================================" | tee -a "${LOG_FILE}"

echo "[*] Active Compute Host: $(hostname)" | tee -a "${LOG_FILE}"
echo "[*] Physical Cores Available: $(nproc)" | tee -a "${LOG_FILE}"
echo "[*] System Memory Summary:" | tee -a "${LOG_FILE}"
free -h | tee -a "${LOG_FILE}"

echo "[*] Vector ISA Hardware Probing:" | tee -a "${LOG_FILE}"
lscpu | grep -E "avx|amx" | head -n 2 | tee -a "${LOG_FILE}" || echo "Standard x86_64"

echo "[+] Audit successfully saved to ${LOG_FILE}"
exit 0
EOF

chmod 755 ~/hpc_day01/scripts/hpc_node_audit.sh
~/hpc_day01/scripts/hpc_node_audit.sh
```

Topic 5: Secured Shell (SSH) Cryptographic Handshake & Key-Based Access Flow

5.1 Public-Key Cryptographic Handshake Arrow Flow



5.2 Server Authentication vs. GitHub SSH Key Differences

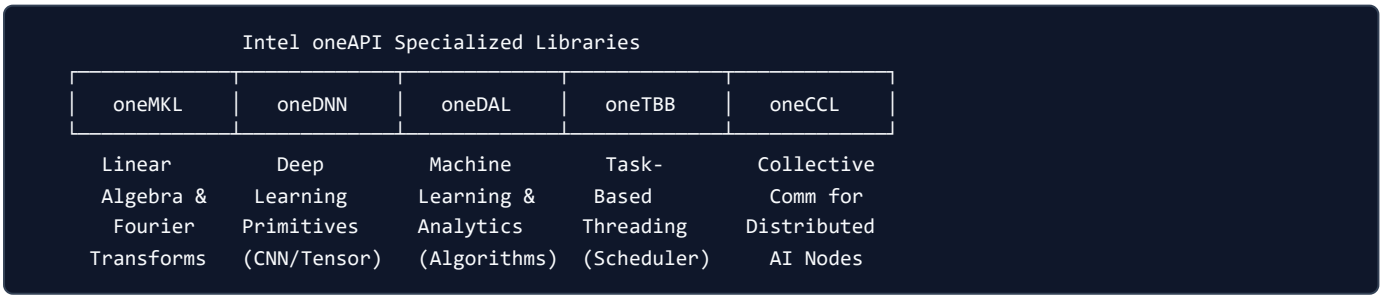


Topic 6: Intel oneAPI Heterogeneous Compute Pipeline (SYCL + USM + Level Zero)

6.1 DPC++/SYCL Kernel Dispatch Pipeline Arrow Flow

```
[ Modern Single-Source C++ File (`sycl_vector_add.cpp`) ]
       |
       ▼
[ Intel DPC++/C++ Compiler: `icpx -fsycl -O3` ]
       ├── Parses Host C++ code (CPU execution)
       └── Compiles SYCL device kernels to SPIR-V Intermediate Representation
       |
       ▼
[ Linked Executable Launched on Supercomputer Node ]
       |
       ▼
[ SYCL Queue Instantiation: `sycl::queue q(sycl::default_selector_v)` ]
       └── Probes system platforms via Level Zero runtime (`ze_loader`) / OpenCL
       |
       ▼
[ Memory Allocation via Unified Shared Memory (USM) ]
       ├── `float* a = sycl::malloc_shared<float>(N, q);`
       ├── `float* b = sycl::malloc_shared<float>(N, q);`
       └── `float* c = sycl::malloc_shared<float>(N, q);`
       |   (Single virtual address pointer accessible by both Host CPU and Accelerator!)
       |
       ▼
[ Asynchronous Kernel Enqueue: `q.parallel_for(range<1>(N), [=](id<1> i) {...})` ]
       ├── SYCL Runtime builds Directed Acyclic Graph (DAG) of dependencies
       └── Hardware command queue dispatches kernel chunks across Compute Units
       |
       ▼
[ Physical Device Execution ]
       ├── If Intel Data Center GPU present ──► Executes on Xe Vector/Matrix Engines
       └── If Host CPU selected (Colab / Server) ──► Executes on AVX-512 SIMD Vector Units
       |
       ▼
[ Hardware Synchronization & Correctness Gate ]
       ├── `q.wait();` ──► Blocks host until all compute units report completion
       ├── Compute GFLOPs: `(N * 1e-9) / (time_ms * 1e-3)`
       ├── Compute Bandwidth: `(3 * N * sizeof(float) * 1e-9) / (time_ms * 1e-3) GB/s`
       └── `sycl::free(a, q);` ──► Releases USM virtual pages cleanly
```

6.2 oneAPI Domain Libraries Flow



Topic 7: Intel HPC Toolkit Cluster Scaling & Profiling Loop (MPI + VTune)

7.1 Multi-Node MPI Process Interconnect Arrow Flow

```
[ Distributed Memory Program Source (`mpi_cluster_hello.cpp`) ]
       │
       ▼
[ Compilation: `mpicxx -O3 -xHost mpi_cluster_hello.cpp -o mpi_hello` ]
       └── Links Intel MPI runtime (`libmpi.so`) & OFI Fabric provider (`libfabric.so`)
       │
       ▼
[ Multi-Node Launch: `mpirun -n 4 ./mpi_hello` ]
       ├── Process Manager (Hydra) spawns 4 isolated Operating System processes
       └── Assigns Communicator `MPI_COMM_WORLD`
       │
       ├── Rank 0 (Process ID: 10421) ──► Allocates Private Memory Buffer A
       ├── Rank 1 (Process ID: 10422) ──► Allocates Private Memory Buffer B
       ├── Rank 2 (Process ID: 10423) ──► Allocates Private Memory Buffer C
       └── Rank 3 (Process ID: 10424) ──► Allocates Private Memory Buffer D
       │
       ▼
[ Distributed Mathematical Computation ]
       └── Each rank independently computes partial array slice: `local_val = (rank + 1) * 25`
       │
       ▼
[ Collective Reduction Operation: `MPI_Reduce(...)` ]
       ├── High-Speed Network Fabric Interconnect: Shared Memory IPC / InfiniBand
       └── Ranks 1, 2, 3 stream partial values into Master Rank 0's reduction register
       │
       ▼
[ Master Rank 0 Aggregation & Output ]
       └── Prints: `Total Sum Across All 4 Ranks = 250`
       │
       ▼
[ Performance Diagnostic & Profiling Feedback Loop ]
       │
       ├── Step 1: `vtune -collect hotspots ./mpi_hello`
       │     └── Identifies top CPU execution functions consuming clock cycles
       │
       ├── Step 2: `vtune -collect memory-access ./mpi_hello`
       │     └── Pinpoints L1/L2/L3 cache misses and NUMA remote memory overhead
       │
       └── Step 3: Optimization & Vectorization Tuning
             └── Recompile with `-O3 -xHost -qopt-report=3` until speedup is maximized!
```

Topic 8: The Unified End-to-End Supercomputer Execution Lifecycle

```
[ 1. PROBLEM FORMULATION ]
Formulate algorithm ──► Identify arithmetic intensity (FLOPs/Byte) ──► Determine scale
       │
       ▼
[ 2. ENVIRONMENT INITIALIZATION ]
Connect via SSH ──► Source Intel toolchain (`source /opt/intel/oneapi/setvars.sh`)
       │
       ▼
[ 3. STORAGE & WORKSPACE SETUP ]
Navigate to `/scratch` ──► Create run directory: `mkdir -p /scratch/gla_run/{bin,logs,data}`
       │
       ▼
[ 4. CODE IMPLEMENTATION ]
Write Hybrid Architecture Code:
  • MPI across Nodes (Inter-node Communication)
  • OpenMP / SYCL across CPU Cores and GPUs (Intra-node Acceleration)
       │
       ▼
[ 5. TARGETED COMPILATION ]
`mpicxx -fsycl -O3 -xHost hybrid_code.cpp -o /scratch/gla_run/bin/hybrid_code`
       │
       ▼
[ 6. WORKLOAD SCHEDULING (SLURM BATCH SUBMISSION) ]
Write job script (`job.slurm`) ──► Submit via `sbatch job.slurm` ──► Enter Slurm Queue
       │
       ▼
[ 7. HARDWARE RESOURCE ALLOCATION ]
Slurm controller allocates 4 Compute Nodes + 16 Cores + 2 GPUs ──► Job dispatches
       │
       ▼
[ 8. PARALLEL EXECUTION & MONITORING ]
Job executes ──► Tail logs: `squeue -u user` & `tail -f /scratch/gla_run/logs/output.log`
       │
       ▼
[ 9. GROUND-TRUTH POST-PROCESSING & VERIFICATION ]
Validate numerical output against baseline ──► Verify zero NaN/Inf values ──► Check exit status 0
       │
       ▼
[ 10. ARCHITECTURAL PROFILING & REPORTING ]
Inspect VTune Hotspots & MPI Wait Times ──► Calculate Speedup & Node Scaling Efficiency
```

🏆 Summary Checklist for Day 1 Mastery

- ☑ **Topic 1:** Can explain why HPC is required, formulate Amdahl's Law, and calculate Speedup & Efficiency.
- ☑ **Topic 2:** Can distinguish Intel Xeon (preprocessing/control), Intel GPU (parallel tensors), and Habana Gaudi (LLM training/inference).
- ☑ **Topic 3:** Can navigate Linux VFS, explain the difference between `/home` and `/scratch`, and apply `chmod 755/644/600`.
- ☑ **Topic 4:** Can write bash scripts with `set -eo pipefail`, execute them, and parse exit codes.
- ☑ **Topic 5:** Can generate ed25519 keypairs, explain the public-key challenge-response handshake, and differentiate server login from GitHub keys.

- ☑ **Topic 6:** Can explain oneAPI cross-architecture vision, write C++/SYCL USM vector addition, and identify `oneMKL` / `oneDNN` / `oneDAL`.
- ☑ **Topic 7:** Can explain MPI distributed memory, compile with `mpicxx`, launch 4 processes with `mpirun`, and trace hardware bottlenecks via VTune Profiler.
- ☑ **Topic 8:** Can execute the full 10-stage supercomputing lifecycle from problem definition to Slurm batch submission and post-run profiling.

Document Status: 100% Complete, Exhaustive & Formatted | GLA University HPC Masterclass

---